HW3

Yi-Ting Tsai

load library

```
library(haven)
```

```
Warning: package 'haven' was built under R version 4.5.3
```

```
library(knitr)
```

```
Warning: package 'knitr' was built under R version 4.5.3
```

Problem 1

a. Download and Read File

```
download.file("https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/AUX_I.xpt", "AUX_
download.file("https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/DEMO_I.xpt", "DEM

aux <- read_xpt("AUX_I.XPT")
demo <- read_xpt("DEMO_I.XPT")

dim(aux)
```

```
[1] 4582   73
```

```
dim(demo)
```

```
[1] 9971   47
```

```
names(aux)
```

```
 [1] "SEQN"     "AUAEXSTS" "AUAEXCMT" "AUQ010"   "AUQ020"   "AUQ020A"
 [7] "AUQ020B"  "AUQ020C"  "AUQ020D"  "AUQ020E"  "AUQ030"   "AUQ040"
[13] "AUQ050"   "AUXOTSPL" "AUXLOEXC" "AUXLOIMC" "AUXLOCOL" "AUXLOABN"
[19] "AUDLOABC" "AUXROTSP" "AUXROEXC" "AUXROIMC" "AUXROCOL" "AUXROABN"
[25] "AUDROABC" "AUXTMEPR" "AUXTPVR"  "AUXTWIDR" "AUXTCOMR" "AUXTMEPL"
[31] "AUXTPVL"  "AUXTWIDL" "AUXTCOML" "AUAEAR"   "AUAMODE"  "AUAFMANL"
[37] "AUAFMANR" "AUXU1K1R" "AUXU500R" "AUXU1K2R" "AUXU2KR"  "AUXU3KR"
[43] "AUXU4KR"  "AUXU6KR"  "AUXU8KR"  "AUXU1K1L" "AUXU500L" "AUXU1K2L"
[49] "AUXU2KL"  "AUXU3KL"  "AUXU4KL"  "AUXU6KL"  "AUXU8KL"  "AUXR1K1R"
[55] "AUXR5CR"  "AUXR1K2R" "AUXR2KR"  "AUXR3KR"  "AUXR4KR"  "AUXR6KR"
[61] "AUXR8KR"  "AUXR1K1L" "AUXR5CL"  "AUXR1K2L" "AUXR2KL"  "AUXR3KL"
[67] "AUXR4KL"  "AUXR6KL"  "AUXR8KL"  "AUAREQC"  "AUATYMTR" "AUALEQC"
[73] "AUATYMTL"
```

```
names(demo)
```

```
 [1] "SEQN"     "SDDSRVYR" "RIDSTATR" "RIAGENDR" "RIDAGEYR" "RIDAGEMN"
 [7] "RIDRETH1" "RIDRETH3" "RIDEXMON" "RIDEXAGM" "DMQMILIZ" "DMQADFC"
[13] "DMDBORN4" "DMDCITZN" "DMDYRSUS" "DMDEDUC3" "DMDEDUC2" "DMDMARTL"
[19] "RIDEXPRG" "SIALANG"  "SIAPROXY" "SIAINTRP" "FIALANG"  "FIAPROXY"
[25] "FIAINTRP" "MIALANG"  "MIAPROXY" "MIAINTRP" "AIALANGA" "DMDHHSIZ"
[31] "DMDFMSIZ" "DMDHHSZA" "DMDHHSZB" "DMDHHSZE" "DMDHRGND" "DMDHRAGE"
[37] "DMDHRBR4" "DMDHREDU" "DMDHRMAR" "DMDHSEDU" "WTINT2YR" "WTMEC2YR"
[43] "SDMVPSU"  "SDMVSTRA" "INDHHIN2" "INDFMIN2" "INDFMPIR"
```

merge into a dataframe

AUXTWIDR: Tympanometric width, right ear

AUXTWIDL: Tympanometric width, left ear

```
attr(aux$AUXTWIDR, "label")
```

```
[1] "Tympanometric width, right ear"
```

```
attr(aux$AUXTWIDL, "label")
```

```
[1] "Tympanometric width, left ear"
```

```r
# check these labels are about Tympanometric
summary(aux$AUXTWIDR)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   8.00   61.00   79.00   84.73   98.00  479.00     433
```

```r
summary(aux$AUXTWIDL)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   2.00   62.00   78.00   85.34   98.00  479.00     479
```

```r
# check the number of NA's
```

```r
aux_s <- aux[, c("SEQN", "AUXTWIDR", "AUXTWIDL")]
names(aux_s) <- c("SEQN", "tymp_r", "tymp_l")

demo_s <- demo[, c("SEQN", "RIAGENDR", "DMDCITZN", "DMDHHSZA", "INDHHIN2")]
names(demo_s) <- c("SEQN", "gender_raw", "citizen_raw", "children_raw", "income_raw")
# get data with only labels we need for AUX and demo
```

```r
data <- merge(
  x = aux_s,
  y = demo_s,
  by = "SEQN",
  all = FALSE
)
# merge AUX and demo
```

```r
dim(data)
```

```
[1] 4582    7
```

```r
# Display the dimensions of the data.frame
```

The merged data frame contained 4592 rows and 7 columns

b. Clean the Variables

```r
table(data$gender_raw, useNA = "always")
```

```
   1    2 <NA>
2176 2406    0
```

```r
table(data$citizen_raw, useNA = "always")
```

```
   1    2    7    9 <NA>
3684  884    8    5    1
```

```r
table(data$children_raw, useNA = "always")
```

```
   0    1    2    3 <NA>
3463  737  317   65    0
```

```r
table(data$income_raw, useNA = "always")
```

```
   1    2    3    4    5    6    7    8    9   10   12   13   14   15   77   99
 101  146  233  262  272  461  439  381  295  244  145   53  458  795   86   63
<NA>
 148
```

```r
summary(data$tymp_r)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   8.00   61.00   79.00   84.73   98.00  479.00     433
```

```r
summary(data$tymp_l)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   2.00   62.00   78.00   85.34   98.00  479.00     479
```

```r
# check if there are any weird numbers like 999
```

```r
data$tymp_r <- as.numeric(data$tymp_r)
data$tymp_l <- as.numeric(data$tymp_l)

data$tymp_r[data$tymp_r %in% c(555, 777, 999)] <- NA
data$tymp_l[data$tymp_l %in% c(555, 777, 999)] <- NA
# recode special numbers as NA

data$tymp_r[!is.na(data$tymp_r) & (data$tymp_r < 0 | data$tymp_r > 499)] <- NA
data$tymp_l[!is.na(data$tymp_l) & (data$tymp_l < 0 | data$tymp_l > 499)] <- NA
# the value must be in the range of 1 ~ 499

summary(data$tymp_r)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   8.00   61.00   79.00   84.73   98.00  479.00     433
```

```r
summary(data$tymp_l)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   2.00   62.00   78.00   85.34   98.00  479.00     479
```

```r
# we check the summary again but we can see that there are no invalid value
```

```r
data$gender_raw[data$gender_raw %in% c(7, 9)] <- NA
data$gender <- factor(
  data$gender_raw,
  levels = c(1, 2),
  labels = c("male", "female")
)
# convert gender to a factor with informative labels.
# male is the first level

data$citizen_raw[data$citizen_raw %in% c(7, 9)] <- NA
data$citizen <- factor(
  data$citizen_raw,
  levels = c(1, 2),
  labels = c("citizen", "noncitizen")
)
```

```r
# convert citizenship status to a factor with informative labels.

data$children_raw[data$children_raw %in% c(7, 9)] <- NA
data$children <- as.numeric(data$children_raw)
data$citizen[!(data$children %in% 0: 3)] <- NA
# convert the number of young children to an ordinary numeric variable.

data$income_raw[data$income_raw %in% c(12, 13, 77, 99)] <- NA
income_midpoint <- c(
  "1"  = 2500,
  "2"  = 7500,
  "3"  = 12500,
  "4"  = 17500,
  "5"  = 22500,
  "6"  = 30000,
  "7"  = 40000,
  "8"  = 50000,
  "9"  = 60000,
  "10" = 70000,
  "14" = 87500,
  "15" = 100000
)
data$income <- unname(income_midpoint[as.character(data$income_raw)])
# convert detailed income categories to approximate dollar midpoints.
# codes 12 and 13 are broad categories that cannot be mapped uniquely.
# codes 77 and 99 are documented missing-value codes.
# all four codes therefore become NA in the continuous variable.

# replace special value in all variables with NA

table(data$gender, useNA = "always")
```

```
  male female   <NA>
  2176   2406      0
```

```r
table(data$citizen, useNA = "always")
```

```
   citizen noncitizen       <NA>
      3684        884         14
```

```r
table(data$children, useNA = "always")
```

```
   0    1    2    3 <NA>
3463  737  317   65    0
```

```r
table(data$income, useNA = "always")
```

```
 2500  7500 12500 17500 22500 30000 40000 50000 60000 70000 87500 1e+05  <NA>
  101   146   233   262   272   461   439   381   295   244   458   795   495
```

The numerical household-income codes were not ordered according to actual income. Codes 12 and 13 could not be uniquely assigned to a detailed income range and were therefore set to NA; codes 77 and 99 were also treated as missing.

c. Fit Poisson Model

```r
modelrp1 <- glm(
  tymp_r ~ gender,
  data = data,
  family = poisson(),
  na.action = na.omit
)
modelrp2 <- glm(
  tymp_r ~ gender + citizen + children + income,
  data = data,
  family = poisson(),
  na.action = na.omit
)
modellp1 <- glm(
  tymp_l ~ gender,
  data = data,
  family = poisson(),
  na.action = na.omit
)
modellp2 <- glm(
  tymp_l ~ gender + citizen + children + income,
```

```
  data = data,
  family = poisson(),
  na.action = na.omit
)
# construct four Poisson models

summary(modelrp1)
```

```
Call:
glm(formula = tymp_r ~ gender, family = poisson(), data = data,
    na.action = na.omit)

Coefficients:
             Estimate Std. Error  z value Pr(>|z|)
(Intercept)  4.434588   0.002468 1796.797  < 2e-16 ***
genderfemale 0.009264   0.003380    2.741  0.00613 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 70970  on 4148  degrees of freedom
Residual deviance: 70963  on 4147  degrees of freedom
  (433 observations deleted due to missingness)
AIC: 96618

Number of Fisher Scoring iterations: 4
```

```
summary(modelrp2)
```

```
Call:
glm(formula = tymp_r ~ gender + citizen + children + income,
    family = poisson(), data = data, na.action = na.omit)

Coefficients:
                    Estimate Std. Error  z value Pr(>|z|)
(Intercept)        4.469e+00  4.332e-03 1031.781  < 2e-16 ***
genderfemale       1.456e-02  3.595e-03    4.050 5.13e-05 ***
citizennoncitizen  4.437e-02  4.613e-03    9.618  < 2e-16 ***
```

```
children          -5.627e-03  2.713e-03   -2.074   0.0381 *
income            -9.062e-07  5.646e-08  -16.051  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 62369  on 3704  degrees of freedom
Residual deviance: 61954  on 3700  degrees of freedom
  (877 observations deleted due to missingness)
AIC: 84858

Number of Fisher Scoring iterations: 4
```

```
summary(modellp1)
```

```
Call:
glm(formula = tymp_l ~ gender, family = poisson(), data = data,
    na.action = na.omit)

Coefficients:
             Estimate Std. Error  z value Pr(>|z|)
(Intercept)  4.439805   0.002473 1795.263  < 2e-16 ***
genderfemale 0.012952   0.003387    3.824 0.000131 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 73307  on 4102  degrees of freedom
Residual deviance: 73293  on 4101  degrees of freedom
  (479 observations deleted due to missingness)
AIC: 98685

Number of Fisher Scoring iterations: 4
```

```
summary(modellp2)
```

```
Call:
```

```
glm(formula = tymp_l ~ gender + citizen + children + income,
    family = poisson(), data = data, na.action = na.omit)

Coefficients:
                    Estimate Std. Error  z value Pr(>|z|)
(Intercept)        4.469e+00  4.346e-03 1028.355  < 2e-16 ***
genderfemale       1.863e-02  3.600e-03    5.176 2.26e-07 ***
citizennoncitizen  2.151e-02  4.648e-03    4.627 3.71e-06 ***
children          -1.707e-02  2.728e-03   -6.257 3.93e-10 ***
income            -6.386e-07  5.637e-08  -11.330  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 64149  on 3664  degrees of freedom
Residual deviance: 63925  on 3660  degrees of freedom
  (917 observations deleted due to missingness)
AIC: 86607

Number of Fisher Scoring iterations: 4
```

produce a table

```r
irr <- function(model, model_name){
  coef_table <- summary(model)$coefficients
  beta <- coef_table[, "Estimate"]
  se <- coef_table[, "Std. Error"]
  p_value <- coef_table[, "Pr(>|z|)"]
  data.frame(
    model = model_name,
    predictor = rownames(coef_table),
    IRR = exp(beta),
    CI_lower = exp(beta - 1.96 * se),
    CI_upper = exp(beta + 1.96 * se),
    p_value = p_value,
    row.names = NULL
  )
}
# use a function to extract exponentiated Poisson coefficients

irr_result <- rbind(
```

```
  irr(modelrp1, "1R - Right ear: gender"),
  irr(modelrp2, "2R - Right ear: adjusted"),
  irr(modellp1, "1L - Left ear: gender"),
  irr(modellp2, "2L - Right ear: adjusted")
)
# combine coefficients from four models

irr_result$`IRR (95% CI)` <- sprintf(
  "%.3f (%.3f, %.3f)",
  irr_result$IRR,
  irr_result$CI_lower,
  irr_result$CI_upper
)
# combine IRR, lower CI and upper CI

irr_result$`p-value` <- ifelse(
  irr_result$p_value < 0.001,
  "<0.001",
  sprintf("%.3f", irr_result$p_value)
)
# format p-vlaues

kable(
  irr_result[, c("model", "predictor", "IRR (95% CI)", "p-value")],
  caption = paste("Incidence rate ratios from the",
                  "Poisson regression models"),
  align = c("l", "l", "c", "c")
)
```

Table 1: Incidence rate ratios from the Poisson regression models

model	predictor	IRR (95% CI)	p-value
1R - Right ear: gender	(Intercept)	84.317 (83.911, 84.726)	<0.001
1R - Right ear: gender	genderfemale	1.009 (1.003, 1.016)	0.006
2R - Right ear: adjusted	(Intercept)	87.301 (86.563, 88.045)	<0.001
2R - Right ear: adjusted	genderfemale	1.015 (1.008, 1.022)	<0.001
2R - Right ear: adjusted	citizennoncitizen	1.045 (1.036, 1.055)	<0.001
2R - Right ear: adjusted	children	0.994 (0.989, 1.000)	0.038
2R - Right ear: adjusted	income	1.000 (1.000, 1.000)	<0.001
1L - Left ear: gender	(Intercept)	84.758 (84.349, 85.170)	<0.001
1L - Left ear: gender	genderfemale	1.013 (1.006, 1.020)	<0.001

model	predictor	IRR (95% CI)	p-value
2L - Right ear: adjusted	(Intercept)	87.312 (86.571, 88.059)	<0.001
2L - Right ear: adjusted	genderfemale	1.019 (1.012, 1.026)	<0.001
2L - Right ear: adjusted	citizennoncitizen	1.022 (1.012, 1.031)	<0.001
2L - Right ear: adjusted	children	0.983 (0.978, 0.988)	<0.001
2L - Right ear: adjusted	income	1.000 (1.000, 1.000)	<0.001

```
# print the coefficient table
```

calculate model statistics

```
pseudo_r2 <- function(model){
  1 - model$deviance / model$null.deviance
}
# define a function to calculate pseudo-R-square

model_list <- list(
  "1R: Right ear, gender" = modelrp1,
  "2R: Right ear, adjusted" = modelrp2,
  "1L: Left ear, gender" = modellp1,
  "2L: Left ear, adjusted" = modellp2
)
# store all four fitted model

model_statistics <- data.frame(
  model = names(model_list),
  N = sapply(model_list, nobs),
  pseudo_R2 = sapply(model_list, pseudo_r2),
  AIC = sapply(model_list, AIC),
  row.names = NULL
)
# calculate sample size, pseudo-R-square, and AIC

names(model_statistics)[3] <- "Deviance pseudo-R²"

kable(model_statistics,
      caption = "Sample sizes and fit statistics for the Poisson models",
      align = c("l", "r", "r", "r"))
```

Table 2: Sample sizes and fit statistics for the Poisson models

model	N	Deviance pseudo-R^2	AIC
1R: Right ear, gender	4149	0.0001058	96618.48
2R: Right ear, adjusted	3705	0.0066514	84857.74
1L: Left ear, gender	4103	0.0001995	98685.15
2L: Left ear, adjusted	3665	0.0034927	86606.80

```
# print the model-statistics table
```

d.Compare males and females in model2L

```
coefficient_2L <- summary(modellp2)$coefficients

gender_beta <- coefficient_2L["genderfemale", "Estimate"]

gender_se <- coefficient_2L["genderfemale", "Std. Error"]

gender_z <- coefficient_2L["genderfemale", "z value"]

gender_p <- coefficient_2L["genderfemale", "Pr(>|z|)"]

gender_irr <- exp(gender_beta)

gender_lower <- exp(gender_beta - 1.96 * gender_se)

gender_upper <- exp(gender_beta + 1.96 * gender_se)

gender_test <- data.frame(
  comparison = "Female vs. Male",
  IRR = round(gender_irr, 3),
  `95% CI lower` = round(gender_lower, 3),
  `95% CI upper` = round(gender_upper, 3),
  `z statistic` = round(gender_z, 3),
  `p-value` = gender_p,
  check.names = FALSE
)
# calculate coefficient, standard error,
# Wald z statistics, Wald p-value, and IRR
```

```
kable(
  gender_test,
  caption = "Model2L test of female-to-male incidence rate ratio"
)
```

Table 3: Model2L test of female-to-male incidence rate ratio

comparison	IRR	95% CI lower	95% CI upper	z statistic	p-value
Female vs. Male	1.019	1.012	1.026	5.176	2e-07

In Model 2L, the female-to-male IRR was 1.019 (95% CI: [1.012, 1.026], z = 5.176, p = 2e-07). Holding the other covariates constant, females had an expected left-ear tympanometric width that was 1.019 times that of males. Because p was less than .05, there was sufficient evidence of a gender difference.

```
modellp2_data <- model.frame(modellp2)
# obtain data used to fit model 2L

prediction_data <- data.frame(
  gender = factor(
    c("male", "female"),
    levels = levels(modellp2_data$gender)
  ),
  citizen = factor(
    c("citizen", "citizen"),
    levels = levels(modellp2_data$citizen)
  ), # use only citizen because the difference we want is gender
  children = rep(
    mean(modellp2_data$children),
    2
  ),
  income = rep(
    mean(modellp2_data$income),
    2
  )
)
# generate new data with different gender

prediction_data
```

```
  gender citizen  children  income
1   male citizen 0.3377899 54320.6
2 female citizen 0.3377899 54320.6
```

```r
pred_link <- predict(
  modellp2,
  newdata = prediction_data,
  type = "link",
  se.fit = TRUE
)
# predict on the log scale

prediction_data$predicted <- exp(pred_link$fit)

prediction_data$lower <- exp(pred_link$fit - 1.96 * pred_link$se.fit)

prediction_data$upper <- exp(pred_link$fit + 1.96 * pred_link$se.fit)

prediction_table <- data.frame(
  gender = prediction_data$gender,
  `adjusted predicted value` = prediction_data$predicted,
  `95% CI lower` = prediction_data$lower,
  `95% CI upper` = prediction_data$upper,
  check.names = FALSE
)
# calculate the original outcome and 95% CI

kable(
  prediction_table,
  caption = "Adjusted predicted left-ear Tympanometric width from model 2L"
)
```

Table 4: Adjusted predicted left-ear Tympanometric width from model 2L

gender	adjusted predicted value	95% CI lower	95% CI upper
male	83.85024	83.39216	84.31084
female	85.42727	84.99682	85.85991

```r
prediction_male <- prediction_data$predicted[
    prediction_data$gender == "male"
  ]
```

```r
prediction_female <- prediction_data$predicted[
    prediction_data$gender == "female"
  ]
# extract predictions for males and females

prediction_diff <- prediction_female - prediction_male

prediction_ratio <- prediction_female / prediction_male
# calculate the predicted difference and ratio

prediction_comp_df <- data.frame(
  statistics = c(
    "adjusted prediction: male",
    "adjusted prediction: female",
    "female minus male",
    "female devided by male",
    "Wald z statistic for equality",
    "Wald p-value for equality"
  ),
  value = c(
    prediction_male,
    prediction_female,
    prediction_diff,
    prediction_ratio,
    gender_z,
    gender_p
  )
)

kable(
  prediction_comp_df,
  caption = "comparison of adjusted predicted values from model 2L"
)
```

Table 5: comparison of adjusted predicted values from model 2L

statistics	value
adjusted prediction: male	83.8502428
adjusted prediction: female	85.4272730
female minus male	1.5770302
female devided by male	1.0188077

statistics	value
Wald z statistic for equality	5.1764513
Wald p-value for equality	0.0000002

With citizenship fixed at Citizen and the continuous covariates held at their Model 2L sample means, the predicted left-ear tympanometric width was 83.85024 for males and 85.42727 for females. The female-minus-male difference was 1.5770302. The test produced z = 2.1764513 and p = 0.0000002; therefore, the predicted values [did/did not] differ significantly between males and females.

Problem 2 - Sakila

load library and database

```
library(DBI)
```

```
Warning: package 'DBI' was built under R version 4.5.3
```

```
library(RSQLite)
```

```
Warning: package 'RSQLite' was built under R version 4.5.3
```

```
library(dplyr)
```

```
Warning: package 'dplyr' was built under R version 4.5.3
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(microbenchmark)
```

```
Warning: package 'microbenchmark' was built under R version 4.5.3
```

```
sakila <- dbConnect(SQLite(), "sakila_master.db")
```

a. Number of customers and percentage of customers are active

Use R for calculation

```
customers <- dbGetQuery(
  sakila,
  "
  SELECT store_id, active
  FROM customer
  "
) # retrieve each customer's store ID and active status from the database.

ans_aR <- customers %>%
  group_by(store_id) %>%
  summarise(
    number_of_customers = n(),
    percent_active = 100 * mean(active == 1, na.rm = TRUE),
    .groups = "drop"
  )
# group customers by store
# count total number of customers
# calculate the percentage of active customers
ans_aR
```

```
# A tibble: 2 x 3
  store_id number_of_customers percent_active
     <int>               <int>          <dbl>
1        1                 326           97.5
2        2                 273           97.4
```

Use SQL for calculation

```
ans_aSQL <- dbGetQuery(
  sakila,
  "
  SELECT
    store_id,
    COUNT(*) AS number_of_customers,
    100.0 * SUM(active) / COUNT(*) AS active_percentage
  FROM CUSTOMER
  GROUP BY store_id
  ORDER BY store_id
  "
) # select store id, number of rows, and sum of active(which is 0 or 1) divided by number of
ans_aSQL
```

```
  store_id number_of_customers active_percentage
1        1                 326          97.54601
2        2                 273          97.43590
```

b. identify the names and country of staff

Use R for calculation

```
staff_df <- dbGetQuery(
  sakila,
  "
  SELECT staff_id, first_name, last_name, address_id
  FROM staff
  "
)

address_df <- dbGetQuery(
  sakila,
  "
  SELECT address_id, city_id
  FROM address
  "
)

city_df <- dbGetQuery(
  sakila,
  "
```

```
  SELECT city_id, country_id
  FROM city
  "
)

country_df <- dbGetQuery(
  sakila,
  "
  SELECT country_id, country
  FROM country
  "
)
# retrieve staff, address, city, country information.
# address_id will be used to connect each staff member to an address.

answer_bR <- staff_df %>%
  left_join(address_df, by = "address_id") %>%
  left_join(city_df, by = "city_id") %>%
  left_join(country_df, by = "country_id") %>%
  transmute(
    staff_id,
    name = paste(first_name, last_name),
    country
  ) %>%
  arrange(staff_id)
# join the four tables in R
answer_bR
```

```
  staff_id         name   country
1        1 Mike Hillyer    Canada
2        2 Jon Stephens Australia
```

Use SQL for calculation

```
ans_bSQL <- dbGetQuery(
  sakila,
  "
  SELECT
    staff.staff_id,
    staff.first_name,
    staff.last_name,
```

```
    country.country
  FROM STAFF AS staff
  JOIN ADDRESS AS address
    ON staff.address_id = address.address_id
  JOIN CITY as city
    ON address.city_id = city.city_id
  JOIN COUNTRY AS country
    ON city.country_id = country.country_id
  ORDER BY staff.staff_id
  "
) # to get country, we have to connect STAFF-ADDRESS-CITY-COUNTRY
ans_bSQL
```

```
  staff_id first_name last_name   country
1        1       Mike   Hillyer    Canada
2        2        Jon  Stephens Australia
```

c. identify the names of the films

Use R for calculation

```
film_df <- dbGetQuery(
  sakila,
  "
  SELECT film_id, title
  FROM film
  "
)

inventory_df <- dbGetQuery(
  sakila,
  "
  SELECT inventory_id, film_id
  FROM inventory
  "
)

rental_df <- dbGetQuery(
  sakila,
  "
  SELECT rental_id, inventory_id
```

```
  FROM rental
  "
)

payment_df <- dbGetQuery(
  sakila,
  "
  SELECT payment_id, rental_id, amount
  FROM payment
  "
)
# retrieve film IDs and titles, inventory records, rental records, payment records

film_revenue_R <- payment_df %>%
  inner_join(rental_df, by = "rental_id") %>%
  inner_join(inventory_df, by = "inventory_id") %>%
  inner_join(film_df, by = "film_id") %>%
  group_by(film_id, title) %>%
  summarise(
    total_rental_value = sum(amount, na.rm = TRUE),
    .groups = "drop"
  )
# combine the tables using payment-rental-inventory-film

answer_cR <- film_revenue_R %>%
  filter(total_rental_value == max(total_rental_value)) %>%
  arrange(title)
# find the film with the largest total rental value

answer_cR
```

```
# A tibble: 1 x 3
  film_id title            total_rental_value
    <int> <chr>                         <dbl>
1     879 TELEGRAPH VOYAGE               232.
```

Use SQL for calculation

```
ans_cSQL <- dbGetQuery(
  sakila,
  "
```

```
  WITH film_revenue AS (
      SELECT
          f.film_id,
          f.title,
          SUM(p.amount) AS total_rental_value
      FROM film AS f
      JOIN inventory AS i
        ON f.film_id = i.film_id
      JOIN rental AS r
        ON i.inventory_id = r.inventory_id
      JOIN payment AS p
        ON r.rental_id = p.rental_id
      GROUP BY f.film_id, f.title
  )
  SELECT
      film_id,
      title,
      total_rental_value
  FROM film_revenue
  WHERE total_rental_value = (
      SELECT MAX(total_rental_value)
      FROM film_revenue
  )
  ORDER BY title
  "
)
# combine the tables using payment-rental-inventory-film
# keep the film whose revenue equals the maximum

ans_cSQL
```

```
  film_id            title total_rental_value
1     879 TELEGRAPH VOYAGE             231.73
```

Benchmark

benchmark for a

```
set.seed(123)
benchmark_a <- microbenchmark(
  R_method = {
```

```
    customers <- dbGetQuery(
      sakila,
      "
      SELECT store_id, active
      FROM customer
      "
    )

    ans_aR <- customers %>%
      group_by(store_id) %>%
      summarise(
        number_of_customers = n(),
        percent_active = 100 * mean(active == 1, na.rm = TRUE),
        .groups = "drop"
      )
  },
  SQL_method = {
    ans_aSQL <- dbGetQuery(
      sakila,
      "
      SELECT
        store_id,
        COUNT(*) AS number_of_customers,
        100.0 * SUM(active) / COUNT(*) AS active_percentage
      FROM CUSTOMER
      GROUP BY store_id
      ORDER BY store_id
      "
    )
  },
  times = 100L
)
# compare two methods in (a) using benchmark
summary(benchmark_a)
```

```
        expr    min      lq     mean median      uq     max neval
1   R_method 3584.5 4083.30 5140.537 4733.0 5571.90 10986.4   100
2 SQL_method  478.1  587.35  765.985  678.2  836.75  2133.4   100
```

benchmark for b

```r
set.seed(123)
benchmark_b <- microbenchmark(
  R_method = {
    staff_df <- dbGetQuery(
      sakila,
      "
      SELECT staff_id, first_name, last_name, address_id
      FROM staff
      "
    )

    address_df <- dbGetQuery(
      sakila,
      "
      SELECT address_id, city_id
      FROM address
      "
    )

    city_df <- dbGetQuery(
      sakila,
      "
      SELECT city_id, country_id
      FROM city
      "
    )

    country_df <- dbGetQuery(
      sakila,
      "
      SELECT country_id, country
      FROM country
      "
    )

    answer_bR <- staff_df %>%
      left_join(address_df, by = "address_id") %>%
      left_join(city_df, by = "city_id") %>%
      left_join(country_df, by = "country_id") %>%
      transmute(
        staff_id,
        name = paste(first_name, last_name),
```

```
        country
      ) %>%
      arrange(staff_id)
  },
  SQL_method = {
    ans_bSQL <- dbGetQuery(
      sakila,
      "
      SELECT
        staff.staff_id,
        staff.first_name,
        staff.last_name,
        country.country
      FROM STAFF AS staff
      JOIN ADDRESS AS address
        ON staff.address_id = address.address_id
      JOIN CITY as city
        ON address.city_id = city.city_id
      JOIN COUNTRY AS country
        ON city.country_id = country.country_id
      ORDER BY staff.staff_id
      "
    )
  },
  times = 100L
)
# compare two methods in (b) using benchmark
summary(benchmark_b)
```

```
        expr    min       lq      mean   median       uq     max neval
1   R_method 9630.6 10277.00 11941.763 10923.55 12435.15 26908.4   100
2 SQL_method  385.4   432.05   603.227   568.90   646.35  2465.1   100
```

benchmark for c

```
set.seed(123)
benchmark_c <- microbenchmark(
  R_method = {
    film_df <- dbGetQuery(
      sakila,
      "
```

```
  SELECT film_id, title
  FROM film
  "
)

inventory_df <- dbGetQuery(
  sakila,
  "
  SELECT inventory_id, film_id
  FROM inventory
  "
)

rental_df <- dbGetQuery(
  sakila,
  "
  SELECT rental_id, inventory_id
  FROM rental
  "
)

payment_df <- dbGetQuery(
  sakila,
  "
  SELECT payment_id, rental_id, amount
  FROM payment
  "
)

film_revenue_R <- payment_df %>%
  inner_join(rental_df, by = "rental_id") %>%
  inner_join(inventory_df, by = "inventory_id") %>%
  inner_join(film_df, by = "film_id") %>%
  group_by(film_id, title) %>%
  summarise(
    total_rental_value = sum(amount, na.rm = TRUE),
    .groups = "drop"
  )

answer_cR <- film_revenue_R %>%
  filter(total_rental_value == max(total_rental_value)) %>%
  arrange(title)
```

```
  },
  SQL_method = {
    ans_cSQL <- dbGetQuery(
      sakila,
      "
      WITH film_revenue AS (
          SELECT
              f.film_id,
              f.title,
              SUM(p.amount) AS total_rental_value
          FROM film AS f
          JOIN inventory AS i
            ON f.film_id = i.film_id
          JOIN rental AS r
            ON i.inventory_id = r.inventory_id
          JOIN payment AS p
            ON r.rental_id = p.rental_id
          GROUP BY f.film_id, f.title
      )
      SELECT
          film_id,
          title,
          total_rental_value
      FROM film_revenue
      WHERE total_rental_value = (
          SELECT MAX(total_rental_value)
          FROM film_revenue
      )
      ORDER BY title
      "
    )
  },
  times = 100L
)
# compare two methods in (c) using benchmark
summary(benchmark_c)
```

```
        expr     min       lq     mean   median       uq      max neval
1   R_method 41.0158 45.60805 50.73747 48.59105 55.86580  71.6805   100
2 SQL_method 65.8522 70.05785 75.25001 72.97545 78.96475 104.0405   100
```

The benchmark results show that the SQL approach had shorter execution time than the

approach that extracted the tables and performed the joins and calculations in R.

Problem 3

import data

```
data_au <- read.csv(
  "au-500/au-500.csv",
  stringsAsFactors = FALSE,
  check.names = FALSE
)
dim(data_au)
```

```
[1] 500  11
```

```
head(data_au)
```

```
  first_name last_name                   company_name               address
1   Rebbecca     Didio         Brandt, Jonathan F Esq         171 E 24th St
2     Stevie     Hallo     Landrum Temporary Services        22222 Acoma St
3     Mariko    Stayer            Inabinet, Macre Esq 534 Schoenborn St #51
4    Gerardo    Woodka       Morris Downing & Sherred     69206 Jackson Ave
5      Mayra      Bena             Buelt, David L Esq     808 Glen Cove Ave
6     Idella  Scotland Artesian Ice & Cold Storage Co      373 Lafayette St
          city state post       phone1       phone2                        email
1       Leith   TAS 7315 03-8174-9123 0458-665-290 rebbecca.didio@didio.com.au
2     Proston   QLD 4613 07-9997-3366 0497-622-620    stevie.hallo@hotmail.com
3       Hamel    WA 6215 08-5558-9019 0427-885-282   mariko_stayer@hotmail.com
4    Talmalmo   NSW 2640 02-6044-4682 0443-795-912  gerardo_woodka@hotmail.com
5   Lane Cove   NSW 1595 02-1455-6085 0453-666-885        mayra.bena@gmail.com
6 Cartmeticup    WA 6316 08-7868-1355 0451-966-921          idella@hotmail.com
                                       web
1       http://www.brandtjonathanfesq.com.au
2 http://www.landrumtemporaryservices.com.au
3         http://www.inabinetmacreesq.com.au
4     http://www.morrisdowningsherred.com.au
5           http://www.bueltdavidlesq.com.au
6 http://www.artesianicecoldstorageco.com.au
```

a. percentage of .com

```
text <- sub("^[a-z]+://", "", tolower(trimws(data_au[["web"]])))
# convert website addresses to lowercase, remove spaces and https://
test <- sub("/.*$", "", text)
# remove everything after domain name
text <- sub(":.*$", "", text)
# remove port number
percent_com <- mean(grepl("\\.com$", text), na.rm = TRUE) * 100
# do not count domain ending with .com.au
percent_com
```

```
[1] 0
```

b. most dommon domain for email

```
email_domain <- sub("^.*@", "", tolower(data_au[["email"]]))
# convert email addresses to lowercase, remove spaces
domain_count <- sort(table(email_domain[
  !is.na(email_domain) &
    email_domain != "" &
    grepl("@", tolower(data_au[["email"]]))
  ]),
  decreasing = TRUE
) # extract everything after @ and count the domains and sort them
names(domain_count)[1]
```

```
[1] "hotmail.com"
```

```
unname(domain_count)[1]
```

```
[1] 114
```

"hotmail.com" is the most common domain of email address

c. proportion of company names

```r
valid_company <- !is.na(data_au[["company_name"]]) & trimws(data_au[["company_name"]]) != ""
# keep only nonmissing and nonempty company names.

has_nonalpha <- grepl(
  "[^A-Za-z,[:space:]]",
  data_au[["company_name"]]
)
# identify names containing any character other than
# letters, commas, and whitespace
prop_nonalpha <- mean(has_nonalpha[valid_company])
prop_nonalpha
```

```
[1] 0.09
```

0.09 of companies have non-alphabetic character

```r
has_nonalpha_no_amp <- grepl(
  "[^A-Za-z,&[:space:]]",
  data_au[["company_name"]]
) # also exclude &

prop_nonalpha_no_amp <- mean(
  has_nonalpha_no_amp[valid_company]
)

prop_nonalpha_no_amp
```

```
[1] 0.008
```

0.008 of companies have non-alphabetic character(exclude &)

d. cell phone format

```r
# define a function that reformats phone number
phone_format <- function(x){
  digits <- gsub("[^0-9]", "", x)
  good <- !is.na(x) & nchar(digits) == 10
  result <- rep(NA_character_, length(x))
```

```
  result[good] <- paste0(
    substr(digits[good], 1, 4), "-",
    substr(digits[good], 5, 7), "-",
    substr(digits[good], 8, 10)
  )
  result
}

cell_phone1 <- phone_format(data_au[["phone1"]])
cell_phone2 <- phone_format(data_au[["phone2"]])
head(cell_phone1, 10)
```

```
 [1] "0381-749-123" "0799-973-366" "0855-589-019" "0260-444-682" "0214-556-
085"
 [6] "0878-681-355" "0865-228-931" "0252-269-402" "0731-849-989" "0868-904-
661"
```

```
head(cell_phone2, 10)
```

```
 [1] "0458-665-290" "0497-622-620" "0427-885-282" "0443-795-912" "0453-666-
885"
 [6] "0451-966-921" "0427-991-688" "0415-961-606" "0411-732-965" "0461-862-
457"
```

```
# print the first ten phone numbers from both columns
```

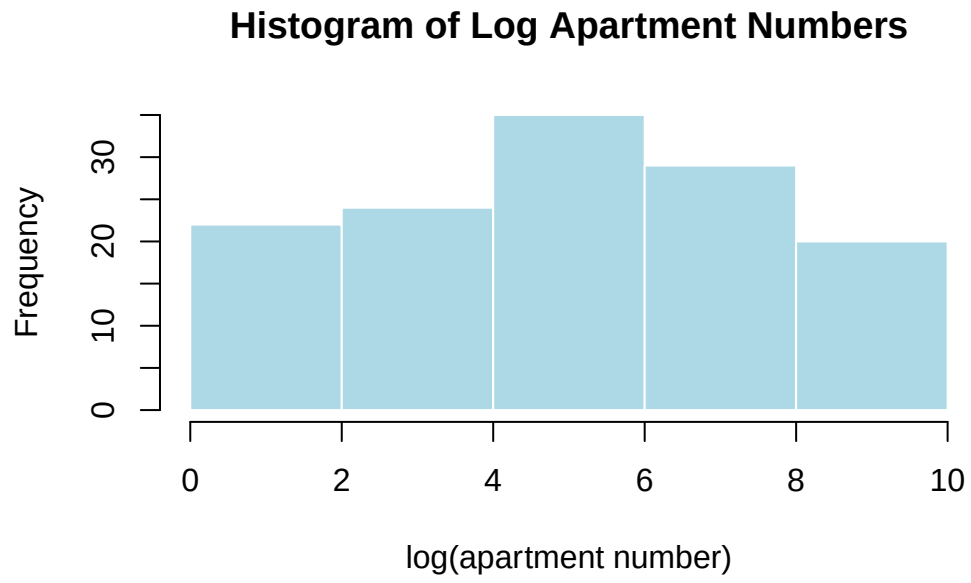
e. histogram of apartment number

```
address <- trimws(data_au[["address"]])
# remove unnecessary spaces from the addresses
has_num_address <- grepl("[0-9]+$", address)
# check the addresses have apartment number or not
apt_num <- rep(NA_real_, length(address))
# create a output vector
apt_num[has_num_address] <- as.numeric(
  sub(".*?([0-9]+)$", "\\1", address[has_num_address])
) # extract the sequence of digits of apartment number
head(data.frame(address, apt_num), 10)
```

```
                address apt_num
1          171 E 24th St      NA
2         22222 Acoma St      NA
3  534 Schoenborn St #51      51
4      69206 Jackson Ave      NA
5      808 Glen Cove Ave      NA
6       373 Lafayette St      NA
7          87 Sylvan Ave      NA
8        60562 Ky Rt 321     321
9           70 S 18th Pl      NA
10     8839 Ventura Blvd      NA
```

```r
valid_apt <- !is.na(apt_num) & apt_num > 0
# zero must be excluded because log(0) is undefined

hist(
  log(apt_num[valid_apt]),
  breaks = "FD",
  main = "Histogram of Log Apartment Numbers",
  xlab = "log(apartment number)",
  col = "lightblue",
  border = "white"
) # draw the histogram of observed digits without NA and 0
```



f. examine Benford's law

```
first_digit <- as.integer(
  substr(as.character(apt_num[!is.na(apt_num) & apt_num > 0]), 1, 1)
)

observed_prop <- prop.table(
  table(factor(first_digit, levels = 1: 9))
)
observed_prop # get first digit of apartment number
```

```
         1          2          3          4          5          6          7
0.09230769 0.14615385 0.12307692 0.10000000 0.10769231 0.13846154 0.06153846
         8          9
0.08461538 0.14615385
```

```
benford_prop <- log10(1 + 1 / 1: 9) # get the distribution of benford's law

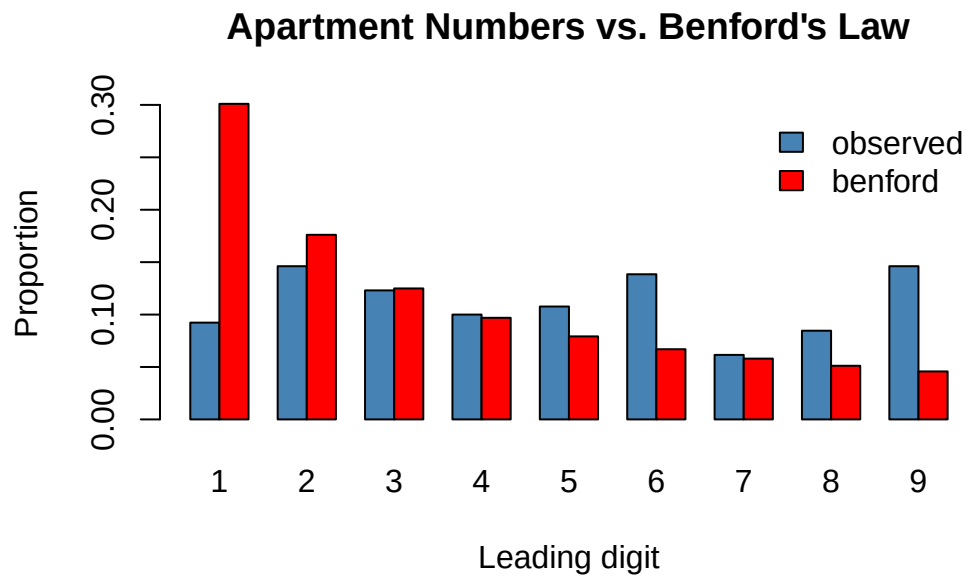
benford_table <- data.frame(
  digit = 1: 9,
  observed = as.numeric(observed_prop),
  benford = benford_prop
) # merge into a dataframe
benford_table
```

```
  digit   observed    benford
1     1 0.09230769 0.30103000
2     2 0.14615385 0.17609126
3     3 0.12307692 0.12493874
4     4 0.10000000 0.09691001
5     5 0.10769231 0.07918125
6     6 0.13846154 0.06694679
7     7 0.06153846 0.05799195
8     8 0.08461538 0.05115252
9     9 0.14615385 0.04575749
```

```
barplot(
  rbind(
    observed = benford_table$observed,
```

```
    benford = benford_table$benford
  ),
  beside = TRUE,
  names.arg = 1: 9,
  xlab = "Leading digit",
  ylab = "Proportion",
  main = "Apartment Numbers vs. Benford's Law",
  col = c("steelblue", "red")
) # drawing barplot of observed digits and Benford's law

legend(
  "topright",
  legend = c("observed", "benford"),
  fill = c("steelblue", "red"),
  bty = "n"
)
```



H_0: The leading digits follow Benford's law.

H_A: The leading digits do not follow Benford's law.

```
benford_test <- chisq.test(
  x = as.numeric(table(factor(first_digit, levels = 1:9))),
```

```
  p = benford_prop
) # doing hypothesis test

benford_test
```

```
    Chi-squared test for given probabilities

data:  as.numeric(table(factor(first_digit, levels = 1:9)))
X-squared = 62.268, df = 8, p-value = 1.67e-10
```